ADVANCED SALES ANALYTICS

A Data-Driven Review of Customer, Product and Revenue Performance

Prepared by

Hermann N'zi Ngenda

Database & Analytics Project

Source schema: gold (fact_sales, dim_customers, dim_products)

May 1, 2026

Executive Summary

This report walks through six analytical exercises run against the gold-layer sales mart of an AdventureWorks-style retail business. The data covers monthly orders from December 2010 through January 2014, joining a fact table of sales transactions to dimension tables for customers and products. Each analysis answers a specific business question — how revenue moves over time, which categories carry the load, who the best customers are, and which products are pulling their weight. The queries are written for Microsoft SQL Server (T-SQL) and use the gold schema.

Three findings stand out. First, the business is concentrated in bikes: that single category accounts for roughly 96% of all revenue, leaving accessories and clothing as small ancillary streams. Second, the customer base skews heavily toward newcomers — about 64% of customers are first-time or short-tenured shoppers, while only 8.5% qualify as VIPs by the spending and lifespan rules in section 5. Third, the trend line shows a strong climb across 2013 followed by a sharp drop in January 2014, which on inspection is a partial-month artifact rather than a real collapse.

The remainder of the document presents each analysis in detail: the question being asked, the SQL used, the output observed, and the takeaway for the business. The full SQL scripts and result exports live alongside this report in the project folder.

Headline Numbers

Metric	Value
Total revenue (lifetime)	$29,356,250
Active months in dataset	38
Total customers	18,484
Bikes share of revenue	96.46%
VIP customers	1,582
Peak month (revenue)	Dec 2013 ($1,874,128)
Products tracked	295

1. Project Background

The project sits on top of a star-schema warehouse already built and curated in a previous stage. The gold layer exposes three production-ready objects:

- gold.fact_sales — order grain transactions with sales amount, quantity, price, order date, customer key and product key.
- gold.dim_customers — one row per customer with name, birthdate and identifying keys.
- gold.dim_products — one row per product with category, subcategory, cost and product name.

All scripts in this report read from these three objects only. No transformations were applied upstream during analysis, so the numbers presented below are reproducible by re-running the scripts in /scripts against the same schema.

Approach

Each analytical question was framed before any SQL was written. The pattern was the same in every case: state the business question, write a query that returns a small, readable result set, export the output to PDF, and write the interpretation. Window functions (SUM OVER, AVG OVER, LAG) carry most of the heavy lifting because the comparisons are nearly always within a partition — by product, by category, by year. CASE expressions handle the segmentation logic.

2. Change Over Time — Monthly Trends

Question

How are sales, customer counts and units sold moving month over month across the life of the business?

Query

```
SELECT
  DATETRUNC(MONTH, order_date) AS order_Date,
  SUM(sales_amount)            AS total_sales,
  COUNT(DISTINCT customer_key) AS total_customers,
  SUM(quantity)                AS total_quantity
FROM gold.fact_sales
WHERE order_date IS NOT NULL
GROUP BY DATETRUNC(MONTH, order_date)
ORDER BY DATETRUNC(MONTH, order_date);
```

What the output shows

The first 25 months — December 2010 through December 2012 — sit in a slow-growth band roughly between $360k and $740k per month, with customer counts under 350. From January 2013 onward the picture changes. Monthly revenue more than doubles within a quarter and continues to climb through the rest of the year, peaking at $1,874,128 in December 2013 with 2,133 distinct customers. January 2014 then crashes to $45,642.

The collapse is not a churn event. The fact that quantity and customer counts also fall to a fraction of December's number, all in lockstep, points to an incomplete or truncated month in the source feed. Any forecasting work should exclude that final row or treat it as partial.

Selected results

Month	Revenue ($)	Customers	Quantity
2011-06	737,793	230	230

Month	Revenue ($)	Customers	Quantity
2012-06	555,142	318	318
2013-06	1,642,948	1,948	5,025
2013-12	1,874,128	2,133	5,520
2014-01	45,642	834	1,970

3. Cumulative Performance

Question

How does the running total of revenue evolve, and what is the moving average price doing alongside it?

Query

```
SELECT
  order_date,
  total_sales,
  SUM(total_sales) OVER (ORDER BY order_date) AS running_total_sales,
  AVG(avg_price)   OVER (ORDER BY order_date) AS moving_average_price
FROM (
  SELECT
    DATETRUNC(month, order_date) AS order_date,
    SUM(sales_amount)            AS total_sales,
    AVG(price)                   AS avg_price
  FROM gold.fact_sales
  WHERE order_date IS NOT NULL
  GROUP BY DATETRUNC(month, order_date)
) t;
```

What the output shows

The running total grows in a textbook S-curve. By the end of 2011 cumulative sales sit at $7.1M, by end of 2012 at $13.0M, and by end of 2013 at $29.3M — meaning more than half of all revenue in the dataset was earned during the final twelve months. The moving average price moves the opposite way. It opens at roughly $3,200 in early 2011 and slides steadily down to $1,745 by January 2014.

Read together, these two lines tell a clear story. The business widened its product mix downmarket — adding lower-priced items and bringing in many more customers — and that change pulled total revenue sharply upward even as the average ticket fell. Volume took over from price.

Selected results

Month	Monthly Sales	Running Total	Moving Avg Price
2011-12	669,395	7,118,507	3,190
2012-12	624,454	12,960,738	2,500

Month	Monthly Sales	Running Total	Moving Avg Price
2013-06	1,642,948	19,612,710	2,076
2013-12	1,874,128	29,305,616	1,792
2014-01	45,642	29,351,258	1,745

4. Product Performance Analysis

Question

For each product, how does the current year compare to its long-run average and to the previous year?

Query

```
WITH yearly_product_sales AS (
  SELECT YEAR(f.order_date) AS order_year,
         p.product_name,
         SUM(f.sales_amount) AS current_sales
  FROM gold.fact_sales f
  LEFT JOIN gold.dim_products p
    ON f.product_key = p.product_key
  WHERE order_date IS NOT NULL
  GROUP BY YEAR(f.order_date), p.product_name
)
SELECT
  order_year, product_name, current_sales,
  AVG(current_sales) OVER (PARTITION BY product_name) AS avg_sales,
  current_sales - AVG(current_sales) OVER (PARTITION BY product_name) AS diff_avg,
  CASE
    WHEN current_sales - AVG(current_sales) OVER (PARTITION BY product_name) > 0 THEN 'Above
Avg'
    WHEN current_sales - AVG(current_sales) OVER (PARTITION BY product_name) < 0 THEN 'Below
Avg'
    ELSE 'Avg'
  END AS avg_change,
  LAG(current_sales) OVER (PARTITION BY product_name ORDER BY order_year) AS py_sales,
  current_sales - LAG(current_sales) OVER (PARTITION BY product_name ORDER BY order_year) AS
yoy_diff
FROM yearly_product_sales
ORDER BY product_name, order_year;
```

What the output shows

The result set holds 271 product-year rows. The pattern is repetitive but informative: the vast majority of items show a 2013 row flagged Above Avg followed by a 2014 row flagged Below Avg. That mirrors the trend in section 2, since the underlying engine is the same — 2013 was the breakout year and 2014 only includes January.

A few products look genuinely strong on their own merits. The Mountain-200 line in Black and Silver crosses $900k in 2013 alone across multiple frame sizes. Touring-1000 in Blue and Yellow each clear $400k

in 2013. Road-150 Red moves over $1M in 2011 across the size range, then disappears from the catalog in later years. These are the items deserving deeper merchandising attention; the long tail of accessories and apparel each contribute under $30k a year and behave as ride-along products to the bike sales.

Top performers, 2013

Product	2013 Sales ($)	vs Avg
Mountain-200 Black-42	970,785	Above Avg
Mountain-200 Silver-38	967,440	Above Avg
Mountain-200 Black-46	947,835	Above Avg
Mountain-200 Black-38	945,540	Above Avg
Mountain-200 Silver-42	902,480	Above Avg
Mountain-200 Silver-46	921,040	Above Avg
Touring-1000 Blue-46	417,200	Above Avg
Road-350-W Yellow-40	416,745	Above Avg

5. Part-to-Whole — Category Contribution

Question

Which product categories carry the revenue line, and by how much?

Query

```
WITH category_Sales AS (
  SELECT category, SUM(sales_amount) AS total_sales
  FROM gold.fact_sales f
  LEFT JOIN gold.dim_products p ON p.product_key = f.product_key
  GROUP BY category
)
SELECT
  category,
  total_sales,
  SUM(total_sales) OVER () AS overall_sales,
  CONCAT(ROUND((CAST(total_sales AS FLOAT) / SUM(total_sales) OVER ()) * 100, 2), '%')
    AS percentage_of_total
FROM category_Sales
ORDER BY total_sales DESC;
```

What the output shows

The numbers leave little room for interpretation. Bikes account for $28.3M of the $29.4M total — 96.46%. Accessories follow at 2.39% and Clothing brings in 1.16%. From a portfolio risk standpoint that is a single

point of failure: any pricing pressure or supply disruption on the bike SKUs would land directly on the top line.

The flip side is opportunity. With more than 18,000 customers walking through the door for a bike, the attach rate on accessories and clothing is low. A focused cross-sell program — helmets, bottles, jerseys bundled at checkout or recommended post-purchase — would be operating against a very large traffic base and small current penetration.

Category	Total Sales ($)	Share of Total
Bikes	28,316,272	96.46%
Accessories	700,262	2.39%
Clothing	339,716	1.16%

6. Data Segmentation

Two segmentation views were produced — one across the product catalog by cost band, and one across the customer base by spending and tenure.

6.1 Product Cost Bands

Query

```
WITH product_segments AS (
  SELECT product_key, product_name, cost,
         CASE
           WHEN cost < 100              THEN 'Below 100'
           WHEN cost BETWEEN 100 AND 500 THEN '100-500'
           WHEN cost BETWEEN 500 AND 1000 THEN '500-1000'
           ELSE 'Above 1000'
         END AS cost_range
  FROM gold.dim_products
)
SELECT cost_range, COUNT(product_key) AS total_products
FROM product_segments
GROUP BY cost_range
ORDER BY total_products DESC;
```

Result

Cost Range	Number of Products
Below 100	110
100-500	101
500-1000	45
Above 1000	39

The catalog is bottom-heavy. Roughly 71% of distinct SKUs sit at a cost below $500, while only 13% live above $1,000. That distribution explains why the moving average price keeps falling in section 3 as more SKUs get sold — the catalog itself was being seeded with cheaper items.

6.2 Customer Behavioral Segments

Question

Group customers by spending behavior and tenure. The rule set: VIP if the customer has more than 12 months of order history and has spent over $5,000; Regular if they have at least one month of history and have spent $5,000 or less; New for everyone else.

Query

```
WITH customer_spending AS (
  SELECT
    c.customer_key,
    SUM(f.sales_amount) AS total_spending,
    MIN(order_date)     AS first_order,
    MAX(order_date)     AS last_order,
    DATEDIFF(month, MIN(order_date), MAX(order_date)) AS lifespan
  FROM gold.fact_sales f
  LEFT JOIN gold.dim_customers c ON f.customer_key = c.customer_key
  GROUP BY c.customer_key
)
SELECT customer_segment, COUNT(customer_key) AS total_customers
FROM (
  SELECT customer_key,
    CASE
      WHEN lifespan > 12 AND total_spending >  5000 THEN 'VIP'
      WHEN lifespan >= 1 AND total_spending <= 5000 THEN 'Regular'
      ELSE 'New'
    END AS customer_segment
  FROM customer_spending
) t
GROUP BY customer_segment
ORDER BY total_customers;
```

Result

Segment	Customers	Share
New	11,888	64.3%
Regular	5,014	27.1%
VIP	1,582	8.6%

The pyramid is steep. Almost two thirds of all customers are either single-month buyers or under a year of history. Roughly one in twelve qualifies as a VIP. From a marketing standpoint, retention work has the highest leverage on the Regular tier — these customers have already proven they will return, but they have not crossed the spending threshold.

7. Customer Report (gold.report_customers)

This script is a reusable view rather than a one-off question. It consolidates per-customer KPIs and assigns each customer to a tenure band, an age bracket, and a behavioral segment in a single output. The intent is to feed downstream BI tools — Power BI, Tableau, or a static dashboard — without forcing each consumer to rewrite the same logic.

Fields produced

- Identity: customer_key, customer_number, customer_name, age, age_group.
- Behavior: customer_segment (VIP / Regular / New).
- Activity: total_orders, total_sales, total_quantity, total_products, last_order_date, lifespan.
- KPIs: recency (months since last order), average_order_value, avg_monthly_spend.

Logic highlights

Age groups are bucketed Under 20, 20-29, 30-39, 40-49, and 50 and above. Average order value is total sales divided by total orders, with a guard for the divide-by-zero case. Average monthly spend is total sales divided by lifespan in months, falling back to total sales when lifespan is zero — that handles single-month customers cleanly.

The view is intended to be created once and queried directly; the CREATE VIEW line at the top of the script is currently commented out so the query can be tested as a SELECT before promotion.

8. Product Report (gold.report_products)

The product report mirrors the customer report on the other dimension. It rolls every product up into a single row carrying lifecycle and revenue health metrics.

Performance tiers

- High-Performer: total sales over $50,000.
- Mid-Range: total sales between $10,000 and $50,000.
- Low-Performer: total sales below $10,000.

Fields produced

- Catalog: product_key, product_name, category, subcategory, cost.
- Activity: total_orders, total_customers, total_sales, total_quantity, lifespan.
- Pricing: avg_selling_price (sales over quantity, rounded to one decimal).
- KPIs: recency_in_months, avg_order_revenue, avg_monthly_revenue.

Unlike the analytical scripts in earlier sections, this view actually persists. The script begins with a guarded DROP and then CREATE VIEW gold.report_products AS, so re-running it cleanly replaces the prior version. Recency is computed against GETDATE(), which means refreshes to the underlying fact table will show up automatically the next time the view is queried.

9. Dashboard Layout

The dashboard is conceived as a single-page operational summary. The intent is for a sales lead or category manager to open it on Monday morning and see, without scrolling, where revenue stands, who the customers are, and which products are leading or lagging. The layout below describes the panels and the SQL each one reads from.

Panel	Visual	Source script
Top KPI strip	Card row	Aggregates from change-over-time-trends_analysis.sql
Revenue over time	Combo line + bar	change-over-time-trends_analysis.sql
Cumulative revenue	Area + secondary axis	cumulative_analysis.sql
Category mix	Donut	part-to-whole-proportional_analysis.sql
Customer segments	Stacked bar	data-segmentation_analysis2.sql
Product cost bands	Histogram	data-segmentation_analysis.sql
Top products YoY	Heatmap	performance_analysis.sql
Customer 360 list	Detail table	report_customers.sql
Product 360 list	Detail table	report_products.sql

KPI Strip

The card row at the top of the dashboard surfaces seven numbers: lifetime revenue, latest-month revenue, latest-month customers, VIP count, bikes share of revenue, average order value, and average monthly spend per active customer. All seven are derived from the queries already documented; nothing new needs to be written for the strip to work.

Filter shelf

Three slicers control the page: order date range, category, and customer segment. With those three in place, each panel can be cross-filtered against the others without writing additional SQL.

10. Key Findings and Recommendations

What the data shows

- Bikes are the business. 96.46% of revenue rides on a single category and the top 30 SKUs in that category drive most of it.
- Growth happened in 2013. Revenue more than doubled year over year, but the underlying customer count grew faster than the average price held — meaning growth came from acquisition rather than basket inflation.
- New customers dominate the file. Almost two thirds of the customer base has limited history. Only 8.6% are VIPs by spend and tenure.
- The catalog is skewed cheap. About seven in ten SKUs cost under $500. Premium products are a minority of the SKU count but pull the heaviest revenue weight.
- The 2014 cliff is a feed artifact. January 2014 sales sit at 2.4% of December's number with proportionally similar drops in customers and quantity. Treat it as partial.

Where to act

- Build a dedicated cross-sell mechanic for accessories and clothing into the bike checkout. With 18,000+ bike buyers and a 1-2% category share, the headroom is large.
- Stand up a Regular-to-VIP nurture program. Regular customers already buy and stay; the gap to VIP is dollars, not loyalty.
- Diversify the bike SKU exposure. The Mountain-200 family alone produces several million in annual revenue. A supplier issue on that line would be visible at the company P&L level.
- Fix the 2014 ingestion gap. Either backfill December 2013 onward or mark the cutoff date in the warehouse so analysts do not consume the partial month as a real signal.

11. Appendix — Files in the Project

File	Description
scripts/change-over-time-trends_analysis.sql	Monthly trend query
scripts/cumulative_analysis.sql	Running totals and moving avg price
scripts/performance_analysis.sql	Yearly product performance vs avg and prior year
scripts/part-to-whole-proportional_analysis.sql	Category share of revenue
scripts/data-segmentation_analysis.sql	Product cost bands
scripts/data-segmentation_analysis2.sql	Customer behavioral segments
scripts/report_customers.sql	Consolidated customer view
scripts/report_products.sql	Consolidated product view (gold.report_products)
result/*.pdf	Result exports for each query
scripts/*.png	Query screenshots for documentation

All queries were authored and tested against Microsoft SQL Server using the gold schema described in section 1. Re-running the scripts in order will reproduce every figure in this report.

— End of report —